

PROJETO E ANÁLISE DE ALGORITMOS – 01A – 2022.2

[Exercício: algoritmos recursivos: especificação, corretude e análise](#)

Exercício: algoritmos recursivos: especificação, corretude e análise?

Fase de avaliação

Fase de configuração	Fase de envio	Fase de avaliação	Fase de cálculo da nota da avaliação	Encerrado
		Fase atual ●		
	Envie seu trabalho Prazo limite dos envios: quinta, 22 set 2022, 23:59 (1394 dias atrás)	Prazo limite da avaliação: quinta, 29 set 2022, 23:59 (1387 dias atrás)		

Seu envio ▼

Você não enviou seu trabalho ainda

Instruções para avaliação ▼

Respostas:

1. a. Quando $n = 0$ sabemos que $b \neq 0$, pois não podemos ter n e b nulos simultaneamente. Como todo valor não nulo elevado a zero vale 1, concluímos que podemos retornar o valor 1 quando $n = 0$. Quando $n = 1$ nós devemos retornar o valor b , pois todo número elevado a 1 vale ele próprio. Para os casos com $n \geq 2$ vimos no enunciado que as sub-instâncias geradas são estritamente menores, e portanto podem ser passadas na chamada recursiva (não precisam entrar no caso base).
- b. O valor de b permanece inalterado nas chamadas recursiva. As sub-instâncias são criadas apenas quando $n \geq 2$, de modo que $\lfloor n/2 \rfloor \geq 1$ e $\lceil n/2 \rceil \geq 1$. Como as funções piso e teto retornam valores inteiros, concluímos que o valor de n recebido em cada chamada recursiva é um inteiro com valor pelo menos 1, satisfazendo assim as pré-condições.
Sobre o tamanho da sub-instância, vimos no enunciado do item anterior que quando $n \geq 2$ os tamanhos $\lfloor n/2 \rfloor$ e $\lceil n/2 \rceil$ das sub-instâncias são estritamente menores que n . Com isso garantimos que as chamadas recursivas sempre recebem sub-instâncias menores.

c. Basta multiplicar as duas sub-soluções retornadas $b^{\lfloor n/2 \rfloor}$ e $b^{\lceil n/2 \rceil}$, pois $b^n = b^{\lfloor n/2 \rfloor} \cdot b^{\lceil n/2 \rceil}$.

d.

```
algoritmo pot_inteira(b,n)
  <pre-cond> e <pos-cond>: descritos no enunciado.
  se n = 0
    retorne 1
  senão, se n = 1
    retorne b
  senão
    s1 = pot_inteira(b, piso(n/2))
    s2 = pot_inteira(b, teto(n/2))
    retorne s1 * s2
```

e. Pelo pseudocódigo extraímos a recorrência $T(n) = 2T(n/2) + \Theta(1) \in \Theta(n)$. Obs.: como o algoritmo força bruta também é $\Theta(n)$, a divisão e conquista não reduziu a complexidade.

f.

```
algoritmo pot_inteira(b,n)
  <pre-cond> e <pos-cond>: descritos no enunciado.
  se n = 0
    retorne 1
  senão, se n = 1
    retorne b
  senão
    s1 = pot_inteira(b, piso(n/2))
    se n é par
      retorne s1 * s1
    senão
      retorne s1 * s1 * b
```

Como fazemos apenas uma chamada recursiva, a recorrência agora vale $T(n) = T(n/2) + \Theta(1) \in \Theta(\log n)$. Conseguimos então uma complexidade menor que o força bruta.

Envios atribuídos para avaliação ▼

Você não recebeu nenhum envio para avaliar



◀ Vídeo: aproximação de recorrências (11min)

Seguir para...

Slides: projeto e corretude de algoritmos recursivos ►

©2020 – Universidade Federal do Ceará – Campus Quixadá.

Todos os direitos reservados.

Av. José de Freitas Queiroz, 5003

Cedro – Quixadá – Ceará CEP: 63902-580

Secretaria do Campus: (88) 3411-9422

📱 Baixar o aplicativo móvel.